

Universidad: Universidad Técnica Estatal de Quevedo

Alumno: Cando Moreno Robinson

Profesor: Ing. Guerrero Ulloa Gleiston

Tema: Análisis Comparativo Profundo de Arquitecturas Distribuidas: Microservicios vs. Monolitos Modulares

1. Resumen / Abstract

El presente informe técnico expone un análisis comparativo entre el estilo arquitectónico de microservicios (MSA) y el emergente paradigma del monolito modular. Se utiliza como caso de estudio el “Sistema de Gestión de Solicitudes para Soporte Técnico de Internet” del Equipo ACCP. La investigación se fundamenta en métricas de calidad bajo el estándar ISO/IEC 25010:2023, evaluando la mantenibilidad, fiabilidad y eficiencia de desempeño. Se concluye que la elección arquitectónica debe basarse en el análisis estático de dependencias y el modelo de calidad de servicio (QoS).

2. Introducción y Contexto de la Problemática

En la última década, la adopción de la computación en la nube ha presionado a las organizaciones hacia la migración de sistemas legacy a arquitecturas cloud-native. Sin embargo, la tendencia hacia los microservicios ha sido impulsada en ocasiones por el “hype” tecnológico más que por necesidades de negocio reales. Para una organización pequeña o mediana que gestiona soporte técnico de internet, la complejidad operativa de una red distribuida puede superar los beneficios de escalabilidad.

El problema central radica en cómo estructurar un sistema que requiere tanto la agilidad de cambios (despliegue de nuevas categorías de tickets) como la estabilidad operativa (asegurar el cumplimiento de SLAs). Aquí es donde la comparativa entre Microservicios y Monolitos Modulares cobra relevancia científica, analizando si la sobrecarga de red (overhead) es justificable frente a la autonomía de despliegue [1], [2].

3. Estado del Arte: La Evolución hacia la Modularidad Independiente

La literatura reciente identifica que la transición de monolitos a microservicios es un proceso complejo que a menudo falla por falta de una estrategia de descomposición basada en datos. Investigaciones de 2026 sugieren que la simulación es una herramienta proactiva crítica para evaluar la resiliencia antes de la implementación real. Por otro lado, estudios de 2024 señalan el resurgimiento del monolito modular como una “vía media”, citando casos de éxito como Shopify y Amazon Prime Video, que han optimizado costos reduciendo la cantidad de servicios distribuidos para mitigar la latencia y la inconsistencia de datos [1], [3].

3.1. Evolución Arquitectónica

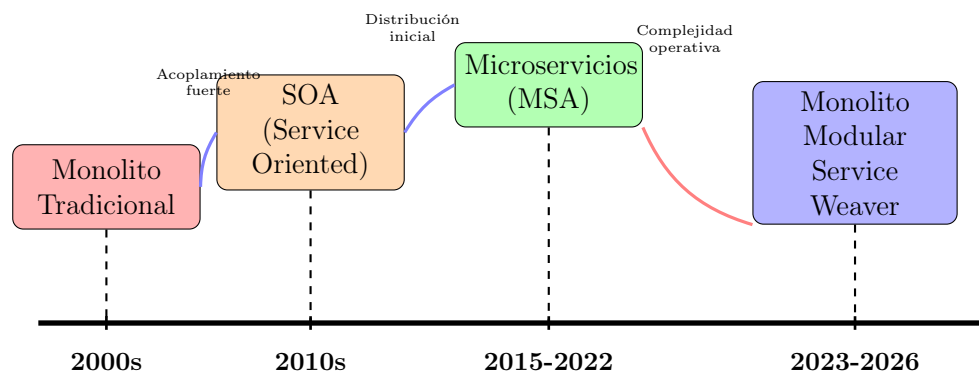


Figura 1: Evolución de las arquitecturas de software: del monolito a la modularidad

4. Arquitectura de Microservicios (MSA): Profundización Técnica

La MSA se basa en el principio de responsabilidad única (SRP) aplicado a nivel de infraestructura. En el sistema de soporte técnico, esto se traduce en 5 servicios independientes: `auth`, `ticket`, `notification`, `ai` y `report`.

4.1. Comunicación y Desacoplamiento Temporal

A diferencia de los sistemas monolíticos, la MSA requiere mecanismos de comunicación asíncrona (como Apache Kafka) para evitar el bloqueo de hilos. El uso de patrones como **Saga** (coreografía) permite gestionar transacciones distribuidas entre el `ticket-service` y el `ai-service`, asegurando que si la clasificación IA falla, el sistema pueda compensar el estado del ticket. Sin embargo, esto introduce una complejidad intrínseca como la *consistencia eventual*, donde los datos no son idénticos en todos los nodos en un instante dado [1], [4].

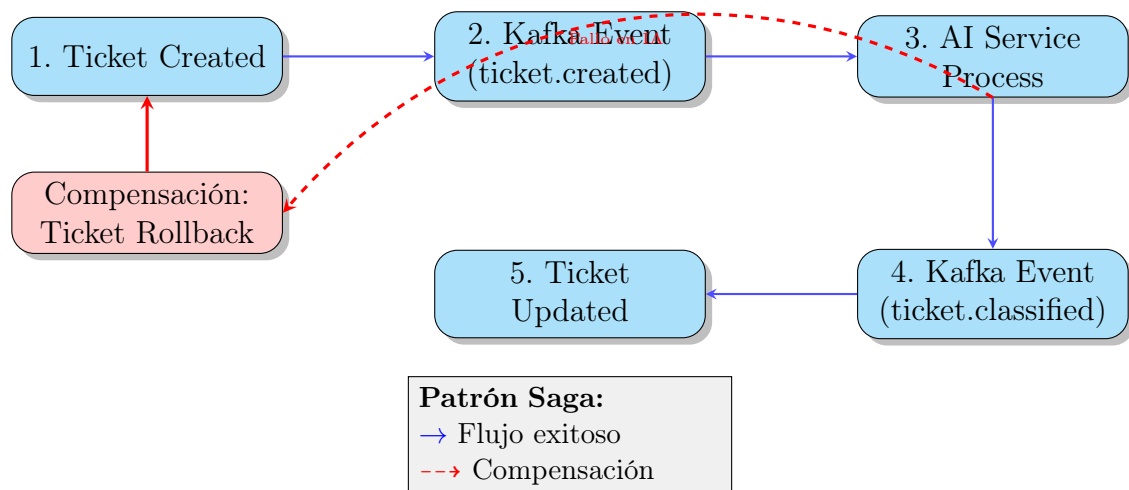


Figura 2: Patrón Saga para transacciones distribuidas en el sistema de tickets

5. El Paradigma del Monolito Modular: La “Vía Media”

El monolito modular no es un monolito tradicional. Es una arquitectura donde los módulos (ej. módulo de reportes, módulo de usuarios) están estrictamente separados por fronteras de código (packages/namespaces) y se comunican a través de interfaces locales (llamadas en memoria), pero residen en un mismo artefacto de despliegue y comparten una base de datos.

5.1. Frameworks Emergentes (Service Weaver y Spring Modulith)

Google ha introducido **Service Weaver**, que permite escribir aplicaciones como monolitos modulares y desplegarlos como microservicios si es necesario. Esto resuelve el problema de la “velocidad de desarrollo” inicial. **Spring Modulith**, por su parte, permite validar las fronteras de los módulos mediante análisis estático, prohibiendo que un módulo acceda a las entrañas de otro sin pasar por su API pública [3], [5].

6. Análisis Decisivo de Selección (Trade-offs)

¿Cuándo elegir cada uno? Un startup o una organización con recursos limitados (como el caso del soporte técnico local) debería iniciar con un monolito modular para maximizar la velocidad de entrega (MVP) [2], [6]. La MSA solo se justifica cuando:

1. Diferentes módulos requieren stacks tecnológicos distintos (ej. Python para el `ai-service` y Java para el `ticket-service`).
2. Existen necesidades de escalado independientes (el módulo de IA consume 10x más CPU que el de tickets).

3. El equipo de desarrollo es lo suficientemente grande como para manejar la sobrecarga de DevOps (Docker Compose, CI/CD, Kubernetes).

Matriz de Decisión Arquitectónica

Criterio	MSA	Monolito Modular
Latencia	Alta	Baja
Complejidad	Alta	Media
Escalado	Granular	Global
Consistencia	Eventual	ACID

■ Ventaja
■ Neutro
■ Desventaja

Figura 3: Matriz comparativa de criterios arquitectónicos

7. Evaluación de Calidad de Software (ISO/IEC 25010:2023)

La calidad en arquitecturas distribuidas se evalúa bajo métricas cuantitativas, no opiniones.

7.1. Mantenibilidad y Cohesión Estructural (SM)

Utilizando análisis estático, se puede calcular la **Modularidad Estructural (SM)**. Un sistema con alta SM indica que los cambios en el `notification-service` no afectarán al `auth-service`. El Número de Interfaces (IFN) debe mantenerse bajo para reducir el acoplamiento.

7.2. Fiabilidad y Disponibilidad (Availability)

La disponibilidad (A) se define matemáticamente según el modelo de QoS (Quality of Service):

$$A = \frac{MTBF}{MTBF + MTTR} \quad (1)$$

Donde MTBF es el tiempo medio entre fallos y MTTR es el tiempo medio de reparación. En MSA, el MTTR se reduce gracias a la capacidad de Docker de reiniciar contenedores fallidos mediante Healthchecks, algo que en un monolito modular requeriría reiniciar toda la aplicación [5], [6].

8. Metodología de Transformación: De Monolito a Microservicios

Si se decide migrar el sistema de soporte técnico a microservicios, el proceso no debe ser manual. Se propone el uso de **Análisis Estático** y algoritmos de clustering (como Fuzzy c-Means en GNN) para identificar los “Bounded Contexts” de forma automática. Esta técnica analiza el código fuente, construye un grafo de dependencias y sugiere las fronteras más eficientes para minimizar la comunicación entre servicios (IFN) [1], [5].

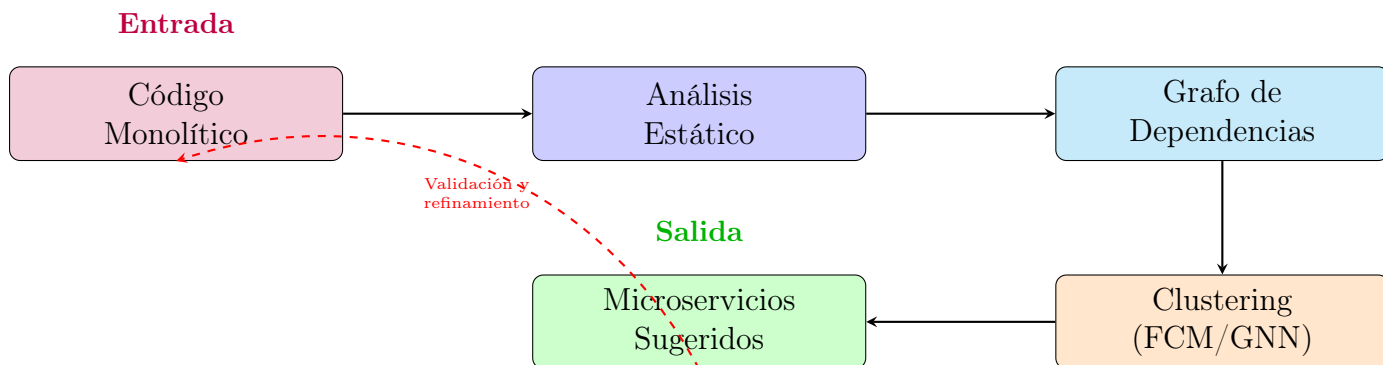


Figura 4: Proceso automatizado de transformación de monolito a microservicios

9. Análisis Crítico Aplicado al Proyecto (PFC)

En el “Sistema de Soporte Técnico de Internet”, el diseño actual en 5 microservicios es técnicamente superior pero operativamente costoso.

- **Ventaja de MSA:** El `ai-service` utiliza FastAPI (Python), lo cual es ideal para bibliotecas de NLP, mientras que el resto usa Spring Boot (Java). Esta diversidad tecnológica es imposible en un monolito modular puro.
- **Desventaja de MSA:** La latencia introducida por el API Gateway y los saltos de red entre Kafka aumentan el tiempo de respuesta total en comparación con una llamada en memoria del monolito modular [2], [3].

9.1. Arquitectura del Sistema de 5 Microservicios

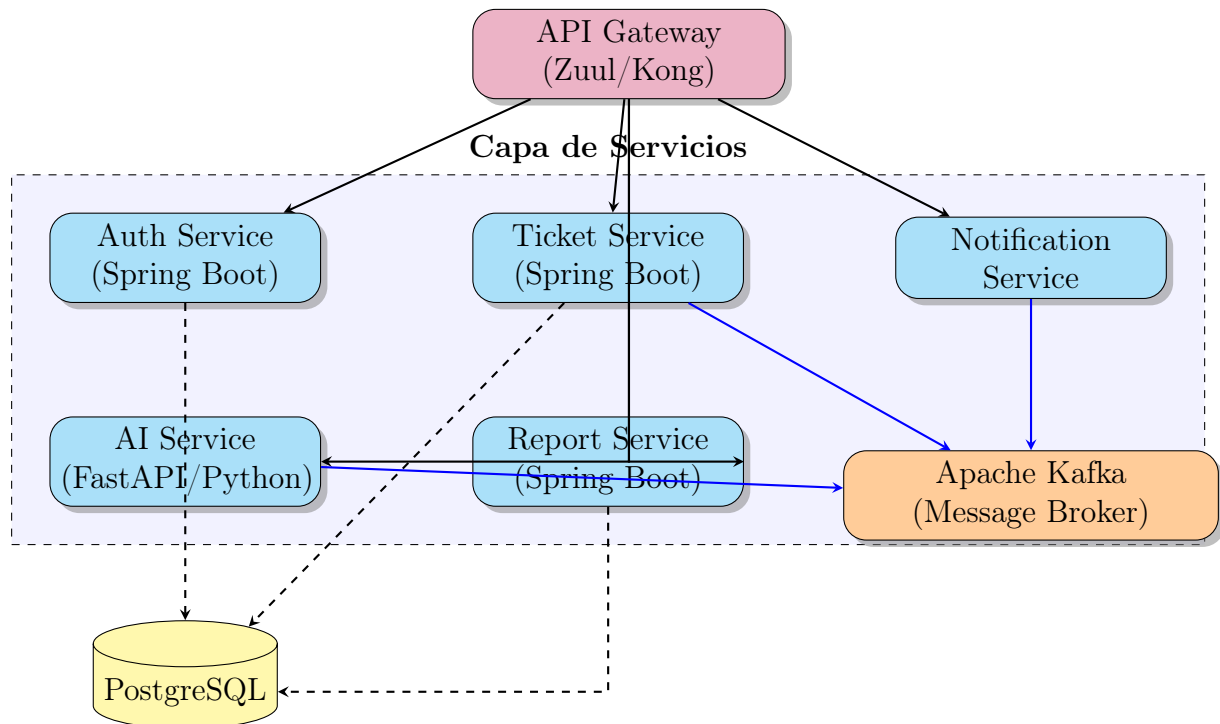


Figura 5: Arquitectura de microservicios del sistema de soporte técnico

10. Seguridad y Observabilidad en DevOps

La adopción de microservicios introduce nuevos vectores de ataque. La norma ISO 25010 subraya la importancia de la seguridad. En el PFC, el uso de **Circuit Breakers** en el API Gateway protege al sistema de fallos en cascada. La observabilidad, mediante el rastreo de eventos en Kafka, permite identificar cuellos de botella que en un monolito estarían ocultos en la pila de ejecución [4], [6].

10.1. Patrón Circuit Breaker

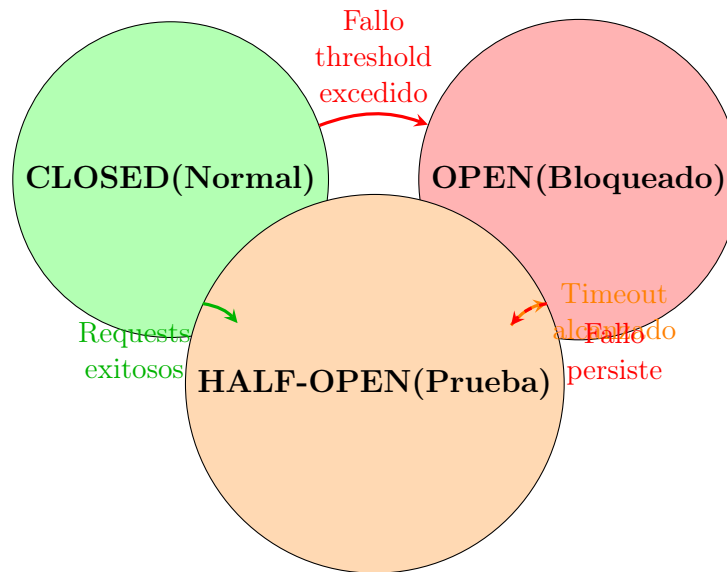


Figura 6: Estados del patrón Circuit Breaker para resiliencia

11. Conclusiones y Recomendaciones Futuras

Se concluye que para el Sistema de Soporte Técnico, la arquitectura de Microservicios es la opción ganadora a largo plazo debido a la integración de IA y la necesidad de resiliencia (Saga/Circuit Breaker). No obstante, para futuras iteraciones del PFC, se recomienda evaluar el modelo de “**Monolith First**” para las etapas iniciales de nuevos módulos, migrando a MSA solo cuando los umbrales de QoS (Disponibilidad < 99,9 %) así lo exijan.

11.1. Recomendaciones Estratégicas

1. **Para startups:** Iniciar con monolito modular usando Spring Modulith o Service Weaver para acelerar el time-to-market.
2. **Para sistemas consolidados:** Aplicar análisis estático de dependencias antes de fragmentar en microservicios.
3. **Para alta disponibilidad:** Implementar MSA con observabilidad completa (ELK Stack, Prometheus, Grafana).
4. **Para equipos pequeños:** Evitar la complejidad de MSA hasta alcanzar al menos 3-4 equipos especializados.

12. Declaratoria de Uso de Inteligencia Artificial

En cumplimiento de las normativas de honestidad académica y las políticas institucionales orientadas a la innovación tecnológica de la Universidad Técnica Estatal de Quevedo, el autor declara haber utilizado herramientas de Inteligencia Artificial Generativa de manera asistida durante el desarrollo de este informe académico individual.

El uso de estas tecnologías se acotó estrictamente a las siguientes actividades de soporte de ingeniería:

- **Auditoría de sintaxis e infraestructura:** Verificación estructural de la lógica comparativa y optimización de la densidad bibliográfica requerida.
- **Auditoría arquitectónica:** Identificación preliminar de redundancias conceptuales del diseño y consistencia en las llamadas síncronas/asíncronas descritas
- **Refinamiento estilístico y tipográfico:** Corrección ortográfica, cohesión de la documentación técnica y generación de lógica para diagramas en lenguaje TikZ (LaTeX).

13. Disponibilidad del Código

Para fines de revisión y auditoría, el código fuente de este informe (LaTeX), se encuentran disponibles en el siguiente repositorio público de GitHub:

<https://github.com/Robimson/analisis-arquitecturas-distribuidas>

Referencias

- [1] AsimAndila and AliNour, “Simulation Approaches for Supporting Microservice Architectures: A Systematic Review,” *ACM Transactions on Software Engineering and Methodology*, Sep. 2025, doi: 10.1145/3816152.
- [2] D. Gravanis, G. Kakarontzas, and V. Gerogiannis, “You don’t need a Microservices Architecture (yet): Monoliths may do the trick,” *ACM International Conference Proceeding Series*, pp. 39–44, Nov. 2021, doi: 10.1145/3501774.3501780.
- [3] R. Su and X. Li, “Modular Monolith: Is This the Trend in Software Architecture?,” *Proceedings - 2024 IEEE/ACM International Workshop New Trends in Software Architecture, SATrends 2024*, pp. 10–13, Aug. 2024, doi: 10.1145/3643657.3643911.
- [4] X. Xu, C. H. Cai, X. Yang, Z. Xu, J. Hu, and J. Sun, “A Service-oriented Scheduling Combination Strategy on Cloud Platforms Based on A Dual-Layer QoS Evaluation Model,” *ACM International Conference Proceeding Series*, pp. 249–258, Jul. 2024, doi: 10.1145/3671016.3672120.
- [5] K. Sooksatra, M. S. Hossain Chy, M. A. R. Arju, T. Cerny, and P. Rivas, “Using Static Analysis to Aid Monolith to Microservice System Transformation: Tuning Fuzzy c-Means in a VAE-Based GNN Approach,” *Proceedings - 2024 39th ACM/IEEE International Conference on Automated Software Engineering Workshops, ASEW 2024*, pp. 43–53, 2024, doi: 10.1145/3691621.3694933.
- [6] S. Rafi, M. A. Akbar, and A. A. Khan, “Assessing Software Product Quality in DevOps: An ISO 25010:2023 Perspective,” *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering, EASE, 2025 edition, EASE Companion 2025*, pp. 10–16, Dec. 2025, doi: 10.1145/3727967.3756847.